

CIS Amazon Elastic Kubernetes Service (EKS) Benchmark

v1.6.0 - 12-23-2024

Terms of Use

Please see the below link for our current terms of use:

<https://www.cisecurity.org/cis-securesuite/cis-securesuite-membership-terms-of-use/>

For information on referencing and/or citing CIS Benchmarks in 3rd party documentation (including using portions of Benchmark Recommendations) please contact CIS Legal (CISLegal@cisecurity.org) and request guidance on copyright usage.

NOTE: It is **NEVER** acceptable to host a CIS Benchmark in **ANY** format (PDF, etc.) on a 3rd party (non-CIS owned) site.

Table of Contents

Terms of Use	1
Table of Contents	2
Overview	5
Important Usage Information	5
Key Stakeholders.....	5
Apply the Correct Version of a Benchmark	6
Exceptions.....	6
Remediation	7
Summary.....	7
Target Technology Details	8
Intended Audience	8
Consensus Guidance.....	9
Typographical Conventions.....	10
Recommendation Definitions	11
Title.....	11
Assessment Status	11
Automated	11
Manual.....	11
Profile.....	11
Description	11
Rationale Statement.....	11
Impact Statement.....	12
Audit Procedure.....	12
Remediation Procedure	12
Default Value	12
References	12
CIS Critical Security Controls® (CIS Controls®).....	12
Additional Information	12
Profile Definitions	13
Acknowledgements.....	14
Recommendations	15
1 Control Plane Components.....	15
2 Control Plane Configuration	15
2.1 Logging.....	16
2.1.1 Enable audit Logs (Automated).....	17
2.1.2 Ensure audit logs are collected and managed (Manual).....	20

3 Worker Nodes	22
3.1 Worker Node Configuration Files	23
3.1.1 Ensure that the kubeconfig file permissions are set to 644 or more restrictive (Automated)	24
3.1.2 Ensure that the kubelet kubeconfig file ownership is set to root:root (Automated)	27
3.1.3 Ensure that the kubelet configuration file has permissions set to 644 or more restrictive (Automated)	30
3.1.4 Ensure that the kubelet configuration file ownership is set to root:root (Automated)	33
3.2 Kubelet.....	36
3.2.1 Ensure that the Anonymous Auth is Not Enabled (Automated)	37
3.2.2 Ensure that the --authorization-mode argument is not set to AlwaysAllow (Automated)	41
3.2.3 Ensure that a Client CA File is Configured (Automated)	45
3.2.4 Ensure that the --read-only-port is disabled (Automated)	48
3.2.5 Ensure that the --streaming-connection-idle-timeout argument is not set to 0 (Automated)	50
3.2.6 Ensure that the --make-iptables-util-chains argument is set to true (Automated)	53
3.2.7 Ensure that the --eventRecordQPS argument is set to 0 or a level which ensures appropriate event capture (Automated)	56
3.2.8 Ensure that the --rotate-certificates argument is not present or is set to true (Automated)	58
3.2.9 Ensure that the RotateKubeletServerCertificate argument is set to true (Automated)	60
4 Policies	63
4.1 RBAC and Service Accounts.....	64
4.1.1 Ensure that the cluster-admin role is only used where required (Automated)	65
4.1.2 Minimize access to secrets (Automated)	67
4.1.3 Minimize wildcard use in Roles and ClusterRoles (Automated)	69
4.1.4 Minimize access to create pods (Automated)	71
4.1.5 Ensure that default service accounts are not actively used. (Automated)	73
4.1.6 Ensure that Service Account Tokens are only mounted where necessary (Automated)	75
4.1.7 Cluster Access Manager API to streamline and enhance the management of access controls within EKS clusters (Automated)	77
4.1.8 Limit use of the Bind, Impersonate and Escalate permissions in the Kubernetes cluster (Manual).....	81
4.2 Pod Security Standards	83
4.2.1 Minimize the admission of privileged containers (Automated)	84
4.2.2 Minimize the admission of containers wishing to share the host process ID namespace (Automated)	87
4.2.3 Minimize the admission of containers wishing to share the host IPC namespace (Automated)	89
4.2.4 Minimize the admission of containers wishing to share the host network namespace (Automated)	91
4.2.5 Minimize the admission of containers with allowPrivilegeEscalation (Automated)	93
4.3 CNI Plugin.....	96
4.3.1 Ensure CNI plugin supports network policies. (Manual)	97
4.3.2 Ensure that all Namespaces have Network Policies defined (Automated)	99
4.4 Secrets Management.....	101
4.4.1 Prefer using secrets as files over secrets as environment variables (Automated)	102
4.4.2 Consider external secret storage (Manual)	104
4.5 General Policies.....	106
4.5.1 Create administrative boundaries between resources using namespaces (Manual)	107
4.5.2 The default namespace should not be used (Automated)	109
5 Managed services	110
5.1 Image Registry and Image Scanning.....	111
5.1.1 Ensure Image Vulnerability Scanning using Amazon ECR image scanning or a third party provider (Automated)	112

5.1.2 Minimize user access to Amazon ECR (Manual)	115
5.1.3 Minimize cluster access to read-only for Amazon ECR (Manual).....	118
5.1.4 Minimize Container Registries to only those approved (Manual)	120
5.2 Identity and Access Management (IAM).....	122
5.2.1 Prefer using dedicated EKS Service Accounts (Automated)	123
5.3 AWS EKS Key Management Service	125
5.3.1 Ensure Kubernetes Secrets are encrypted using Customer Master Keys (CMKs) managed in AWS KMS (Manual)	126
5.4 Cluster Networking.....	128
5.4.1 Restrict Access to the Control Plane Endpoint (Automated)	129
5.4.2 Ensure clusters are created with Private Endpoint Enabled and Public Access Disabled (Automated)	132
5.4.3 Ensure clusters are created with Private Nodes (Automated)	134
5.4.4 Ensure Network Policy is Enabled and set as appropriate (Automated)	136
5.4.5 Encrypt traffic to HTTPS load balancers with TLS certificates (Manual)	138
5.5 Authentication and Authorization.....	139
5.5.1 Manage Kubernetes RBAC users with AWS IAM Authenticator for Kubernetes or Upgrade to AWS CLI v1.16.156 or greater (Manual)	140
Appendix: Summary Table	142
Appendix: CIS Controls v7 IG 1 Mapped Recommendations	147
Appendix: CIS Controls v7 IG 2 Mapped Recommendations	149
Appendix: CIS Controls v7 IG 3 Mapped Recommendations	152
Appendix: CIS Controls v7 Unmapped Recommendations.....	155
Appendix: CIS Controls v8 IG 1 Mapped Recommendations	156
Appendix: CIS Controls v8 IG 2 Mapped Recommendations	158
Appendix: CIS Controls v8 IG 3 Mapped Recommendations	161
Appendix: CIS Controls v8 Unmapped Recommendations.....	164
Appendix: Change History	165

Overview

All CIS Benchmarks™ (Benchmarks) focus on technical configuration settings used to maintain and/or increase the security of the addressed technology, and they should be used in **conjunction** with other essential cyber hygiene tasks like:

- Monitoring the base operating system and applications for vulnerabilities and quickly updating with the latest security patches.
- End-point protection (Antivirus software, Endpoint Detection and Response (EDR), etc.).
- Logging and monitoring user and system activity.

In the end, the Benchmarks are designed to be a key **component** of a comprehensive cybersecurity program.

Important Usage Information

All Benchmarks are available free for non-commercial use from the [CIS Website](#). They can be used to manually assess and remediate systems and applications. In lieu of manual assessment and remediation, there are several tools available to assist with assessment:

- [CIS Configuration Assessment Tool \(CIS-CAT® Pro Assessor\)](#)
- [CIS Benchmarks™ Certified 3rd Party Tooling](#)

These tools make the hardening process much more scalable for large numbers of systems and applications.

NOTE: Some tooling focuses only on the Benchmark Recommendations that can be fully automated (skipping ones marked **Manual**). It is important that **ALL** Recommendations (**Automated** and **Manual**) be addressed since all are important for properly securing systems and are typically in scope for audits.

Key Stakeholders

Cybersecurity is a collaborative effort, and cross functional cooperation is imperative within an organization to discuss, test, and deploy Benchmarks in an effective and efficient way. The Benchmarks are developed to be best practice configuration guidelines applicable to a wide range of use cases. In some organizations, exceptions to specific Recommendations will be needed, and this team should work to prioritize the problematic Recommendations based on several factors like risk, time, cost, and labor. These exceptions should be properly categorized and documented for auditing purposes.

Apply the Correct Version of a Benchmark

Benchmarks are developed and tested for a specific set of products and versions and applying an incorrect Benchmark to a system can cause the resulting pass/fail score to be incorrect. This is due to the assessment of settings that do not apply to the target systems. To assure the correct Benchmark is being assessed:

- **Deploy the Benchmark applicable to the way settings are managed in the environment:** An example of this is the Microsoft Windows family of Benchmarks, which have separate Benchmarks for Group Policy, Intune, and Stand-alone systems based upon how system management is deployed. Applying the wrong Benchmark in this case will give invalid results.
- **Use the most recent version of a Benchmark:** This is true for all Benchmarks, but especially true for cloud technologies. Cloud technologies change frequently and using an older version of a Benchmark may have invalid methods for auditing and remediation.

Exceptions

The guidance items in the Benchmarks are called recommendations and not requirements, and exceptions to some of them are expected and acceptable. The Benchmarks strive to be a secure baseline, or starting point, for a specific technology, with known issues identified during Benchmark development are documented in the Impact section of each Recommendation. In addition, organizational, system specific requirements, or local site policy may require changes as well, or an exception to a Recommendation or group of Recommendations (e.g. A Benchmark could Recommend that a Web server not be installed on the system, but if a system's primary purpose is to function as a Webserver, there should be a documented exception to this Recommendation for that specific server).

In the end, exceptions to some Benchmark Recommendations are common and acceptable, and should be handled as follows:

- The reasons for the exception should be reviewed cross-functionally and be well documented for audit purposes.
- A plan should be developed for mitigating, or eliminating, the exception in the future, if applicable.
- If the organization decides to accept the risk of this exception (not work toward mitigation or elimination), this should be documented for audit purposes.

It is the responsibility of the organization to determine their overall security policy, and which settings are applicable to their unique needs based on the overall risk profile for the organization.

Remediation

CIS has developed [Build Kits](#) for many technologies to assist in the automation of hardening systems. Build Kits are designed to correspond to Benchmark's "Remediation" section, which provides the manual remediation steps necessary to make that Recommendation compliant to the Benchmark.

When remediating systems (changing configuration settings on deployed systems as per the Benchmark's Recommendations), please approach this with caution and test thoroughly.

The following is a reasonable remediation approach to follow:

- CIS Build Kits, or internally developed remediation methods should never be applied to production systems without proper testing.
- Proper testing consists of the following:
 - Understand the configuration (including installed applications) of the targeted systems. Various parts of the organization may need different configurations (e.g., software developers vs standard office workers).
 - Read the Impact section of the given Recommendation to help determine if there might be an issue with the targeted systems.
 - Test the configuration changes with representative lab system(s). If issues arise during testing, they can be resolved prior to deploying to any production systems.
 - When testing is complete, initially deploy to a small sub-set of production systems and monitor closely for issues. If there are issues, they can be resolved prior to deploying more broadly.
 - When the initial deployment above is completed successfully, iteratively deploy to additional systems and monitor closely for issues. Repeat this process until the full deployment is complete.

Summary

Using the Benchmarks Certified tools, working as a team with key stakeholders, being selective with exceptions, and being careful with remediation deployment, it is possible to harden large numbers of deployed systems in a cost effective, efficient, and safe manner.

NOTE: As previously stated, the PDF versions of the CIS Benchmarks™ are available for free, non-commercial use on the [CIS Website](#). All other formats of the CIS Benchmarks™ (MS Word, Excel, and [Build Kits](#)) are available for CIS [SecureSuite®](#) members.

CIS-CAT® Pro is also available to CIS [SecureSuite®](#) members.

Target Technology Details

This document provides prescriptive guidance for running Amazon Elastic Kubernetes Service (EKS) following recommended security controls. This benchmark only includes controls which can be modified by an end user of Amazon EKS.

To obtain the latest version of this guide, please visit www.cisecurity.org. If you have questions, comments, or have identified ways to improve this guide, please write us at support@cisecurity.org.

Intended Audience

This document is intended for cluster administrators, security specialists, auditors, and any personnel who plan to develop, deploy, assess, or secure solutions that incorporate Amazon EKS using managed and self-managed nodes.

Customers using Amazon EKS on AWS Fargate are not responsible for node management. Hence, this document is not scoped for Amazon EKS on AWS Fargate customers.

Consensus Guidance

This CIS Benchmark™ was created using a consensus review process comprised of a global community of subject matter experts. The process combines real world experience with data-based information to create technology specific guidance to assist users to secure their environments. Consensus participants provide perspective from a diverse set of backgrounds including consulting, software development, audit and compliance, security research, operations, government, and legal.

Each CIS Benchmark undergoes two phases of consensus review. The first phase occurs during initial Benchmark development. During this phase, subject matter experts convene to discuss, create, and test working drafts of the Benchmark. This discussion occurs until consensus has been reached on Benchmark recommendations. The second phase begins after the Benchmark has been published. During this phase, all feedback provided by the Internet community is reviewed by the consensus team for incorporation in the Benchmark. If you are interested in participating in the consensus process, please visit <https://workbench.cisecurity.org/>.

Typographical Conventions

The following typographical conventions are used throughout this guide:

Convention	Meaning
<code>Stylized Monospace font</code>	Used for blocks of code, command, and script examples. Text should be interpreted exactly as presented.
<code>Monospace font</code>	Used for inline code, commands, UI/Menu selections or examples. Text should be interpreted exactly as presented.
<code><Monospace font in brackets></code>	Text set in angle brackets denote a variable requiring substitution for a real value.
<i>Italic font</i>	Used to reference other relevant settings, CIS Benchmarks and/or Benchmark Communities. Also, used to denote the title of a book, article, or other publication.
Bold font	Additional information or caveats things like Notes , Warnings , or Cautions (usually just the word itself and the rest of the text normal).

Recommendation Definitions

The following defines the various components included in a CIS recommendation as applicable. If any of the components are not applicable it will be noted, or the component will not be included in the recommendation.

Title

Concise description for the recommendation's intended configuration.

Assessment Status

An assessment status is included for every recommendation. The assessment status indicates whether the given recommendation can be automated or requires manual steps to implement. Both statuses are equally important and are determined and supported as defined below:

Automated

Represents recommendations for which assessment of a technical control can be fully automated and validated to a pass/fail state. Recommendations will include the necessary information to implement automation.

Manual

Represents recommendations for which assessment of a technical control cannot be fully automated and requires all or some manual steps to validate that the configured state is set as expected. The expected state can vary depending on the environment.

Profile

A collection of recommendations for securing a technology or a supporting platform. Most benchmarks include at least a Level 1 and Level 2 Profile. Level 2 extends Level 1 recommendations and is not a standalone profile. The Profile Definitions section in the benchmark provides the definitions as they pertain to the recommendations included for the technology.

Description

Detailed information pertaining to the setting with which the recommendation is concerned. In some cases, the description will include the recommended value.

Rationale Statement

Detailed reasoning for the recommendation to provide the user a clear and concise understanding on the importance of the recommendation.

Impact Statement

Any security, functionality, or operational consequences that can result from following the recommendation.

Audit Procedure

Systematic instructions for determining if the target system complies with the recommendation.

Remediation Procedure

Systematic instructions for applying recommendations to the target system to bring it into compliance according to the recommendation.

Default Value

Default value for the given setting in this recommendation, if known. If not known, either not configured or not defined will be applied.

References

Additional documentation relative to the recommendation.

CIS Critical Security Controls® (CIS Controls®)

The mapping between a recommendation and the CIS Controls is organized by CIS Controls version, Safeguard, and Implementation Group (IG). The Benchmark in its entirety addresses the CIS Controls safeguards of (v7) "5.1 - Establish Secure Configurations" and (v8) "4.1 - Establish and Maintain a Secure Configuration Process" so individual recommendations will not be mapped to these safeguards.

Additional Information

Supplementary information that does not correspond to any other field but may be useful to the user.

Profile Definitions

The following configuration profiles are defined by this Benchmark:

- **Level 1**

Level 1 Configuration Profile

- **Level 2**

Extends Level 1

Acknowledgements

This Benchmark exemplifies the great things a community of users, vendors, and subject matter experts can accomplish through consensus collaboration. The CIS community thanks the entire consensus team with special recognition to the following individuals who contributed greatly to the creation of this guide:

Thanks to Authors: Paavan Mistry and Randall Mowen
with special thanks to contributors: Rory McCune and Tony Wilwerding

Authors:

Paavan Mistry
Randall Mowen

Editor:

Randall Mowen

Contributors

Mark Larinde
Rory McCune
Tony Wilwerding
James Stocks
Daniel Burns
Joe Bowbeer

Recommendations

1 Control Plane Components

Security is a shared responsibility between AWS and the Amazon EKS customer. [The shared responsibility model](#) describes this as security of the cloud and security in the cloud:

Security of the cloud – AWS is responsible for protecting the infrastructure that runs AWS services in the AWS Cloud. For Amazon EKS, AWS is responsible for the Kubernetes control plane, which includes the control plane nodes and etcd database. Third-party auditors regularly test and verify the effectiveness of our security as part of the [AWS compliance programs](#). To learn about the compliance programs that apply to Amazon EKS, see [AWS Services in Scope by Compliance Program](#).

Security in the cloud – Your responsibility includes the following areas.

- The security configuration of the data plane, including the configuration of the security groups that allow traffic to pass from the Amazon EKS control plane into the customer VPC
- The configuration of the worker nodes and the containers themselves
- The worker node guest operating system (including updates and security patches)
 - Amazon EKS follows the shared responsibility model for CVEs and security patches on managed node groups. Because managed nodes run the Amazon EKS-optimized AMIs, Amazon EKS is responsible for building patched versions of these AMIs when bugs or issues are reported and we are able to publish a fix. However, customers are responsible for deploying these patched AMI versions to your managed node groups.
- Other associated application software:
 - Setting up and managing network controls, such as firewall rules
 - Managing platform-level identity and access management, either with or in addition to IAM
- The sensitivity of your data, your company's requirements, and applicable laws and regulations

AWS is responsible for securing the control plane, though you might be able to configure certain options based on your requirements. Section 2 of this Benchmark addresses these configurations.

2 Control Plane Configuration

This section contains recommendations for Amazon EKS control plane logging configuration. Customers are able to configure logging for control plane in Amazon EKS.

2.1 Logging

2.1.1 Enable audit Logs (Automated)

Profile Applicability:

- Level 1

Description:

Control plane logs provide visibility into operation of the EKS Control plane component systems. The API server audit logs record all accepted and rejected requests in the cluster. When enabled via EKS configuration the control plane logs for a cluster are exported to a CloudWatch Log Group for persistence.

Rationale:

Audit logs enable visibility into all API server requests from authentic and anonymous sources. Stored log data can be analyzed manually or with tools to identify and understand anomalous or negative activity and lead to intelligent remediations.

Impact:

Enabling control plane logs, including API server audit logs for Amazon EKS clusters, significantly strengthens our security posture by providing detailed visibility into all API requests, thereby reducing our attack surface. By exporting these logs to a CloudWatch Log Group, we ensure persistent storage and facilitate both manual and automated analysis to quickly identify and remediate anomalous activities. While this configuration might slightly impact usability or performance due to the overhead of logging, the enhanced security and compliance benefits far outweigh these drawbacks, making it a critical component of our security strategy.

Audit:

From Console:

1. For each EKS Cluster in each region;
2. Go to 'Amazon EKS' > 'Clusters' > 'CLUSTER_NAME' > 'Configuration' > 'Logging'.
3. This will show the control plane logging configuration:

```
API server: Enabled / Disabled
Audit: Enabled / Disabled
Authenticator: Enabled / Disabled
Controller manager: Enabled / Disabled
Scheduler: Enabled / Disabled
```

4. Ensure that all options are set to 'Enabled'.

From CLI:

```
# For each EKS Cluster in each region;
export CLUSTER_NAME=<your cluster name>
export REGION_CODE=<your region_code>
aws eks describe-cluster --name ${CLUSTER_NAME} --region ${REGION_CODE} --
query 'cluster.logging.clusterLogging'
```

Remediation:

From Console:

1. For each EKS Cluster in each region;
2. Go to 'Amazon EKS' > 'Clusters' > " > 'Configuration' > 'Logging'.
3. Click 'Manage logging'.
4. Ensure that all options are toggled to 'Enabled'.

```
API server: Enabled
Audit: Enabled
Authenticator: Enabled
Controller manager: Enabled
Scheduler: Enabled
```

5. Click 'Save Changes'.

From CLI:

```
# For each EKS Cluster in each region;
aws eks update-cluster-config \
  --region '${REGION_CODE}' \
  --name '${CLUSTER_NAME}' \
  --logging
'{"clusterLogging":[{"types":["api","audit","authenticator","controllerManager","scheduler"],"enabled":true}]}'
```

Default Value:

Control Plane Logging is disabled by default.

```
API server: Disabled
Audit: Disabled
Authenticator: Disabled
Controller manager: Disabled
Scheduler: Disabled
```

References:

1. <https://kubernetes.io/docs/tasks/debug-application-cluster/audit/>
2. <https://aws.github.io/aws-eks-best-practices/detective/>
3. <https://docs.aws.amazon.com/eks/latest/userguide/control-plane-logs.html>
4. <https://docs.aws.amazon.com/eks/latest/userguide/logging-using-cloudtrail.html>

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	8.1 <u>Establish and Maintain an Audit Log Management Process</u> Establish and maintain an audit log management process that defines the enterprise's logging requirements. At a minimum, address the collection, review, and retention of audit logs for enterprise assets. Review and update documentation annually, or when significant enterprise changes occur that could impact this Safeguard.			
v8	8.2 <u>Collect Audit Logs</u> Collect audit logs. Ensure that logging, per the enterprise's audit log management process, has been enabled across enterprise assets.			
v7	6.2 <u>Activate audit logging</u> Ensure that local logging has been enabled on all systems and networking devices.			
v7	6.3 <u>Enable Detailed Logging</u> Enable system logging to include detailed information such as an event source, date, user, timestamp, source addresses, destination addresses, and other useful elements.			

2.1.2 Ensure audit logs are collected and managed (Manual)

Profile Applicability:

- Level 1

Description:

Ensure that audit logs are collected and managed in accordance with the enterprise's audit log management process across all Kubernetes components.

Rationale:

Audit logs provide visibility into the activities occurring within a Kubernetes cluster, enabling the detection and investigation of security incidents and policy violations. Proper collection and management of audit logs are essential for maintaining an audit trail and ensuring compliance with security policies.

Impact:

Implementing comprehensive audit logging may require additional storage and processing resources. Care must be taken to ensure that logs are properly secured and managed to avoid any potential security risks associated with log data.

Audit:

1. Verify audit logging is enabled for Kubernetes components:

```
kubectl get --raw /api/v1/nodes/${NODE_NAME}/proxy/configz | jq  
' .kubeletConfig.auditPolicy'
```

2. Ensure the audit logs are being collected and sent to a centralized logging system:

```
kubectl get --raw /api/v1/nodes/${NODE_NAME}/proxy/stats/summary | jq  
' .auditLogs'
```

3. Verify that the audit logs are being monitored and managed according to the enterprise's audit log management process.

Remediation:

1. Create or update the audit-policy.yaml to specify the audit logging configuration:

```
apiVersion: audit.k8s.io/v1
kind: Policy
rules:
- level: Metadata
  resources:
  - group: ""
    resources: ["pods"]
```

2. Apply the audit policy configuration to the cluster:

```
kubectl apply -f <path-to-audit-policy>.yaml
```

3. Ensure audit logs are forwarded to a centralized logging system like CloudWatch, Elasticsearch, or another log management solution:

```
kubectl create configmap cluster-audit-policy --from-file=audit-policy.yaml -
n kube-system
kubectl apply -f - <<EOF
apiVersion: v1
kind: Pod
metadata:
  name: audit-logging
  namespace: kube-system
spec:
  containers:
  - name: audit-log-forwarder
    image: my-log-forwarder-image
    volumeMounts:
    - mountPath: /etc/kubernetes/audit
      name: audit-config
  volumes:
  - name: audit-config
    configMap:
      name: cluster-audit-policy
EOF
```

Default Value:

By default, Kubernetes does not enable detailed audit logging. Configuration is required to enable and manage audit logs.

References:

1. <https://kubernetes.io/docs/tasks/debug-application-cluster/audit/>
2. <https://kubernetes.io/docs/tasks/debug-application-cluster/audit/#audit-policy>

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	8.1 <u>Establish and Maintain an Audit Log Management Process</u> Establish and maintain an audit log management process that defines the enterprise's logging requirements. At a minimum, address the collection, review, and retention of audit logs for enterprise assets. Review and update documentation annually, or when significant enterprise changes occur that could impact this Safeguard.			
v8	8.2 <u>Collect Audit Logs</u> Collect audit logs. Ensure that logging, per the enterprise's audit log management process, has been enabled across enterprise assets.			
v7	6.2 <u>Activate audit logging</u> Ensure that local logging has been enabled on all systems and networking devices.			
v7	6.3 <u>Enable Detailed Logging</u> Enable system logging to include detailed information such as an event source, date, user, timestamp, source addresses, destination addresses, and other useful elements.			

3 Worker Nodes

This section consists of security recommendations for the components that run on Amazon EKS worker nodes.

3.1 Worker Node Configuration Files

This section covers recommendations for configuration files on Amazon EKS worker nodes.

3.1.1 Ensure that the kubeconfig file permissions are set to 644 or more restrictive (Automated)

Profile Applicability:

- Level 1

Description:

If kubelet is running, and if it is configured by a kubeconfig file, ensure that the proxy kubeconfig file has permissions of 644 or more restrictive.

Rationale:

The **kubelet** kubeconfig file controls various parameters of the **kubelet** service in the worker node. You should restrict its file permissions to maintain the integrity of the file. The file should be writable by only the administrators on the system.

It is possible to run **kubelet** with the kubeconfig parameters configured as a Kubernetes ConfigMap instead of a file. In this case, there is no proxy kubeconfig file.

Impact:

Ensuring that the kubeconfig file permissions are set to 644 or more restrictive significantly strengthens the security posture of the Kubernetes environment by preventing unauthorized modifications. This restricts write access to the kubeconfig file, ensuring only administrators can alter crucial kubelet configurations, thereby reducing the risk of malicious alterations that could compromise the cluster's integrity.

However, this configuration may slightly impact usability, as it limits the ability for non-administrative users to make quick adjustments to the kubelet settings. Administrators will need to balance security needs with operational flexibility, potentially requiring adjustments to workflows for managing kubelet configurations.

Audit:

Method 1

SSH to the worker nodes

To check to see if the Kubelet Service is running:

```
sudo systemctl status kubelet
```

The output should return **Active: active (running) since..**

Run the following command on each node to find the appropriate kubeconfig file:

```
ps -ef | grep kubelet
```

The output of the above command should return something similar to **--kubeconfig /var/lib/kubelet/kubeconfig** which is the location of the kubeconfig file.

Run this command to obtain the kubeconfig file permissions:

```
stat -c %a /var/lib/kubelet/kubeconfig
```

The output of the above command gives you the kubeconfig file's permissions. Verify that if a file is specified and it exists, the permissions are **644** or more restrictive.

Method 2

Create and Run a Privileged Pod.

You will need to run a pod that is privileged enough to access the host's file system. This can be achieved by deploying a pod that uses the hostPath volume to mount the node's file system into the pod.

Here's an example of a simple pod definition that mounts the root of the host to /host within the pod:

```
apiVersion: v1
kind: Pod
metadata:
  name: file-check
spec:
  volumes:
  - name: host-root
    hostPath:
      path: /
      type: Directory
  containers:
  - name: nsenter
    image: busybox
    command: ["sleep", "3600"]
    volumeMounts:
    - name: host-root
      mountPath: /host
    securityContext:
      privileged: true
```

Save this to a file (e.g., file-check-pod.yaml) and create the pod:

```
kubectl apply -f file-check-pod.yaml
```

Once the pod is running, you can exec into it to check file permissions on the node:

```
kubectl exec -it file-check -- sh
```

Now you are in a shell inside the pod, but you can access the node's file system through the /host directory and check the permission level of the file:

```
ls -l /host/var/lib/kubelet/kubeconfig
```

Verify that if a file is specified and it exists, the permissions are **644** or more restrictive.

Remediation:

Run the below command (based on the file location on your system) on the each worker node. For example,

```
chmod 644 <kubeconfig file>
```

Default Value:

See the AWS EKS documentation for the default value.

References:

1. <https://kubernetes.io/docs/admin/kube-proxy/>

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	3.3 <u>Configure Data Access Control Lists</u> Configure data access control lists based on a user's need to know. Apply data access control lists, also known as access permissions, to local and remote file systems, databases, and applications.			
v7	5.2 <u>Maintain Secure Images</u> Maintain secure images or templates for all systems in the enterprise based on the organization's approved configuration standards. Any new system deployment or existing system that becomes compromised should be imaged using one of those images or templates.			

3.1.2 Ensure that the kubelet kubeconfig file ownership is set to root:root (Automated)

Profile Applicability:

- Level 1

Description:

If **kubelet** is running, ensure that the file ownership of its kubeconfig file is set to **root:root**.

Rationale:

The kubeconfig file for **kubelet** controls various parameters for the **kubelet** service in the worker node. You should set its file ownership to maintain the integrity of the file. The file should be owned by **root:root**.

Impact:

None

Audit:

Method 1

SSH to the worker nodes

To check to see if the Kubelet Service is running:

```
sudo systemctl status kubelet
```

The output should return **Active: active (running) since..**

Run the following command on each node to find the appropriate kubeconfig file:

```
ps -ef | grep kubelet
```

The output of the above command should return something similar to **--kubeconfig /var/lib/kubelet/kubeconfig** which is the location of the kubeconfig file.

Run this command to obtain the kubeconfig file ownership:

```
stat -c %U:%G /var/lib/kubelet/kubeconfig
```

The output of the above command gives you the kubeconfig file's ownership. Verify that the ownership is set to **root:root**.

Method 2

Create and Run a Privileged Pod.

You will need to run a pod that is privileged enough to access the host's file system.

This can be achieved by deploying a pod that uses the hostPath volume to mount the node's file system into the pod.

Here's an example of a simple pod definition that mounts the root of the host to /host within the pod:

```
apiVersion: v1
kind: Pod
metadata:
  name: file-check
spec:
  volumes:
  - name: host-root
    hostPath:
      path: /
      type: Directory
  containers:
  - name: nsenter
    image: busybox
    command: ["sleep", "3600"]
    volumeMounts:
    - name: host-root
      mountPath: /host
    securityContext:
      privileged: true
```

Save this to a file (e.g., file-check-pod.yaml) and create the pod:

```
kubectl apply -f file-check-pod.yaml
```

Once the pod is running, you can exec into it to check file ownership on the node:

```
kubectl exec -it file-check -- sh
```

Now you are in a shell inside the pod, but you can access the node's file system through the /host directory and check the ownership of the file:

```
ls -l /host/var/lib/kubelet/kubeconfig
```

The output of the above command gives you the kubeconfig file's ownership. Verify that the ownership is set to **root:root**.

Remediation:

Run the below command (based on the file location on your system) on each worker node.

For example,

```
chown root:root <proxy kubeconfig file>
```

Default Value:

See the AWS EKS documentation for the default value.

References:

1. <https://kubernetes.io/docs/admin/kube-proxy/>

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	3.3 <u>Configure Data Access Control Lists</u> Configure data access control lists based on a user's need to know. Apply data access control lists, also known as access permissions, to local and remote file systems, databases, and applications.			
v7	5.2 <u>Maintain Secure Images</u> Maintain secure images or templates for all systems in the enterprise based on the organization's approved configuration standards. Any new system deployment or existing system that becomes compromised should be imaged using one of those images or templates.			

3.1.3 Ensure that the kubelet configuration file has permissions set to 644 or more restrictive (Automated)

Profile Applicability:

- Level 1

Description:

Ensure that if the kubelet refers to a configuration file with the `--config` argument, that file has permissions of 644 or more restrictive.

Rationale:

The kubelet reads various parameters, including security settings, from a config file specified by the `--config` argument. If this file is specified you should restrict its file permissions to maintain the integrity of the file. The file should be writable by only the administrators on the system.

Impact:

None.

Audit:

Method 1

First, SSH to the relevant worker node:

To check to see if the Kubelet Service is running:

```
sudo systemctl status kubelet
```

The output should return **Active: active (running) since..**

Run the following command on each node to find the appropriate Kubelet config file:

```
ps -ef | grep kubelet
```

The output of the above command should return something similar to `--config /etc/kubernetes/kubelet/config.json` which is the location of the Kubelet config file.

Run the following command:

```
stat -c %a /etc/kubernetes/kubelet/config.json
```

The output of the above command is the Kubelet config file's permissions. Verify that the permissions are **644** or more restrictive.

Method 2

Create and Run a Privileged Pod.

You will need to run a pod that is privileged enough to access the host's file system. This can be achieved by deploying a pod that uses the hostPath volume to mount the node's file system into the pod.

Here's an example of a simple pod definition that mounts the root of the host to /host within the pod:

```
apiVersion: v1
kind: Pod
metadata:
  name: file-check
spec:
  volumes:
  - name: host-root
    hostPath:
      path: /
      type: Directory
  containers:
  - name: nsenter
    image: busybox
    command: ["sleep", "3600"]
    volumeMounts:
    - name: host-root
      mountPath: /host
    securityContext:
      privileged: true
```

Save this to a file (e.g., file-check-pod.yaml) and create the pod:

```
kubectl apply -f file-check-pod.yaml
```

Once the pod is running, you can exec into it to check file permissions on the node:

```
kubectl exec -it file-check -- sh
```

Now you are in a shell inside the pod, but you can access the node's file system through the /host directory and check the permission level of the file:

```
ls -l /host/etc/kubernetes/kubelet/config.json
```

Verify that if a file is specified and it exists, the permissions are **644** or more restrictive.

Remediation:

Run the following command (using the config file location identified in the Audit step)

```
chmod 644 /etc/kubernetes/kubelet/config.json
```

Default Value:

See the AWS EKS documentation for the default value.

References:

1. <https://kubernetes.io/docs/tasks/administer-cluster/kubelet-config-file/>

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	3.3 <u>Configure Data Access Control Lists</u> Configure data access control lists based on a user's need to know. Apply data access control lists, also known as access permissions, to local and remote file systems, databases, and applications.			
v7	5.2 <u>Maintain Secure Images</u> Maintain secure images or templates for all systems in the enterprise based on the organization's approved configuration standards. Any new system deployment or existing system that becomes compromised should be imaged using one of those images or templates.			

3.1.4 Ensure that the kubelet configuration file ownership is set to root:root (Automated)

Profile Applicability:

- Level 1

Description:

Ensure that if the kubelet refers to a configuration file with the `--config` argument, that file is owned by root:root.

Rationale:

The kubelet reads various parameters, including security settings, from a config file specified by the `--config` argument. If this file is specified you should restrict its file permissions to maintain the integrity of the file. The file should be writable by only the administrators on the system.

Impact:

None

Audit:

Method 1

First, SSH to the relevant worker node:

To check to see if the Kubelet Service is running:

```
sudo systemctl status kubelet
```

The output should return **Active: active (running) since..**

Run the following command on each node to find the appropriate Kubelet config file:

```
ps -ef | grep kubelet
```

The output of the above command should return something similar to `--config /etc/kubernetes/kubelet/config.json` which is the location of the Kubelet config file.

Run the following command:

```
stat -c %U:%G /etc/kubernetes/kubelet/config.json
```

The output of the above command is the Kubelet config file's ownership. Verify that the ownership is set to **root:root**

Method 2

Create and Run a Privileged Pod.

You will need to run a pod that is privileged enough to access the host's file system.

This can be achieved by deploying a pod that uses the hostPath volume to mount the node's file system into the pod.

Here's an example of a simple pod definition that mounts the root of the host to /host within the pod:

```
apiVersion: v1
kind: Pod
metadata:
  name: file-check
spec:
  volumes:
  - name: host-root
    hostPath:
      path: /
      type: Directory
  containers:
  - name: nsenter
    image: busybox
    command: ["sleep", "3600"]
    volumeMounts:
    - name: host-root
      mountPath: /host
    securityContext:
      privileged: true
```

Save this to a file (e.g., file-check-pod.yaml) and create the pod:

```
kubectl apply -f file-check-pod.yaml
```

Once the pod is running, you can exec into it to check file ownership on the node:

```
kubectl exec -it file-check -- sh
```

Now you are in a shell inside the pod, but you can access the node's file system through the /host directory and check the ownership of the file:

```
ls -l /etc/kubernetes/kubelet/config.json
```

The output of the above command gives you the azure.json file's ownership. Verify that the ownership is set to **root:root**.

Remediation:

Run the following command (using the config file location identified in the Audit step)

```
chown root:root /etc/kubernetes/kubelet/config.json
```

Default Value:

See the AWS EKS documentation for the default value.

References:

1. <https://kubernetes.io/docs/admin/kube-proxy/>

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	3.3 <u>Configure Data Access Control Lists</u> Configure data access control lists based on a user's need to know. Apply data access control lists, also known as access permissions, to local and remote file systems, databases, and applications.			
v7	5.2 <u>Maintain Secure Images</u> Maintain secure images or templates for all systems in the enterprise based on the organization's approved configuration standards. Any new system deployment or existing system that becomes compromised should be imaged using one of those images or templates.			

3.2 Kubelet

Kubelets can accept configuration via a configuration file and in some cases via command line arguments. It is important to note that parameters provided as command line arguments will override their counterpart parameters in the configuration file (see `--config` details in the [Kubelet CLI Reference](#) for more info, where you can also find out which configuration parameters can be supplied as a command line argument).

With this in mind, it is important to check for the existence of command line arguments as well as configuration file entries when auditing Kubelet configuration.

Firstly, SSH to each node and execute the following command to find the Kubelet process:

```
ps -ef | grep kubelet
```

The output of the above command provides details of the active Kubelet process, from which we can see the command line arguments provided to the process. Also note the location of the configuration file, provided with the `--config` argument, as this will be needed to verify configuration. The file can be viewed with a command such as `more` or `less`, like so:

```
sudo less /path/to/kubelet-config.json
```

This config file could be in JSON or YAML format depending on your distribution.

3.2.1 Ensure that the Anonymous Auth is Not Enabled (Automated)

Profile Applicability:

- Level 1

Description:

Disable anonymous requests to the Kubelet server.

Rationale:

When enabled, requests that are not rejected by other configured authentication methods are treated as anonymous requests. These requests are then served by the Kubelet server. You should rely on authentication to authorize access and disallow anonymous requests.

Impact:

This configuration might have a slight impact on usability for users who rely on anonymous access for certain functions or quick troubleshooting. Additionally, there might be a minimal performance overhead due to the added authentication steps for each request.

Audit:

Audit Method 1:

Kubelets can accept configuration via a configuration file and in some cases via command line arguments. It is important to note that parameters provided as command line arguments will override their counterpart parameters in the configuration file (see `--config` details in the [Kubelet CLI Reference](#) for more info, where you can also find out which configuration parameters can be supplied as a command line argument).

With this in mind, it is important to check for the existence of command line arguments as well as configuration file entries when auditing Kubelet configuration.

Firstly, SSH to each node and execute the following command to find the Kubelet process:

```
ps -ef | grep kubelet
```

The output of the above command provides details of the active Kubelet process, from which we can see the command line arguments provided to the process. Also note the location of the configuration file, provided with the `--config` argument, as this will be needed to verify configuration. The file can be viewed with a command such as `more` or `less`, like so:

```
sudo less /path/to/kubelet-config.json
```

Verify that Anonymous Authentication is not enabled. This may be configured as a command line argument to the kubelet service with `--anonymous-auth=false` or in the kubelet configuration file via `"authentication": { "anonymous": { "enabled": false } }`.

Audit Method 2:

It is also possible to review the running configuration of a Kubelet via the `/configz` endpoint of the Kubernetes API. This can be achieved using `kubectl` to proxy your requests to the API.

Discover all nodes in your cluster by running the following command:

```
kubectl get nodes
```

Next, initiate a proxy with `kubectl` on a local port of your choice. In this example we will use 8080:

```
kubectl proxy --port=8080
```

With this running, in a separate terminal run the following command for each node:

```
export NODE_NAME=my-node-name  
curl http://localhost:8080/api/v1/nodes/${NODE_NAME}/proxy/configz
```

The curl command will return the API response which will be a JSON formatted string representing the Kubelet configuration.

Verify that Anonymous Authentication is not enabled checking that `"authentication": { "anonymous": { "enabled": false } }` is in the API response.

Remediation:

Remediation Method 1:

If configuring via the Kubelet config file, you first need to locate the file.

To do this, SSH to each node and execute the following command to find the kubelet process:

```
ps -ef | grep kubelet
```

The output of the above command provides details of the active kubelet process, from which we can see the location of the configuration file provided to the kubelet service with the `--config` argument. The file can be viewed with a command such as `more` or `less`, like so:

```
sudo less /path/to/kubelet-config.json
```

Disable Anonymous Authentication by setting the following parameter:

```
"authentication": { "anonymous": { "enabled": false } }
```

Remediation Method 2:

If using executable arguments, edit the kubelet service file on each worker node and ensure the below parameters are part of the `KUBELET_ARGS` variable string.

For systems using `systemd`, such as the Amazon EKS Optimised Amazon Linux or Bottlerocket AMIs, then this file can be found at `/etc/systemd/system/kubelet.service.d/10-kubelet-args.conf`. Otherwise, you may need to look up documentation for your chosen operating system to determine which service manager is configured:

```
--anonymous-auth=false
```

For Both Remediation Steps:

Based on your system, restart the `kubelet` service and check the service status.

The following example is for operating systems using `systemd`, such as the Amazon EKS Optimised Amazon Linux or Bottlerocket AMIs, and invokes the `systemctl` command. If `systemctl` is not available then you will need to look up documentation for your chosen operating system to determine which service manager is configured:

```
systemctl daemon-reload
systemctl restart kubelet.service
systemctl status kubelet -l
```

Default Value:

See the EKS documentation for the default value.

References:

1. <https://kubernetes.io/docs/reference/command-line-tools-reference/kubelet/>
2. <https://kubernetes.io/docs/reference/access-authn-authz/kubelet-authn-authz/#kubelet-authentication>
3. <https://kubernetes.io/docs/reference/config-api/kubelet-config.v1beta1/>

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	5.3 <u>Disable Dormant Accounts</u> Delete or disable any dormant accounts after a period of 45 days of inactivity, where supported.			
v7	14.6 <u>Protect Information through Access Control Lists</u> Protect all information stored on systems with file system, network share, claims, application, or database specific access control lists. These controls will enforce the principle that only authorized individuals should have access to the information based on their need to access the information as a part of their responsibilities.			

3.2.2 Ensure that the `--authorization-mode` argument is not set to `AlwaysAllow` (Automated)

Profile Applicability:

- Level 1

Description:

Do not allow all requests. Enable explicit authorization.

Rationale:

Kubelets can be configured to allow all authenticated requests (even anonymous ones) without needing explicit authorization checks from the apiserver. You should restrict this behavior and only allow explicitly authorized requests.

Impact:

Unauthorized requests will be denied.

Audit:

Audit Method 1:

Kubelets can accept configuration via a configuration file and in some cases via command line arguments. It is important to note that parameters provided as command line arguments will override their counterpart parameters in the configuration file (see `--config` details in the [Kubelet CLI Reference](#) for more info, where you can also find out which configuration parameters can be supplied as a command line argument).

With this in mind, it is important to check for the existence of command line arguments as well as configuration file entries when auditing Kubelet configuration.

Firstly, SSH to each node and execute the following command to find the Kubelet process:

```
ps -ef | grep kubelet
```

The output of the above command provides details of the active Kubelet process, from which we can see the command line arguments provided to the process. Also note the location of the configuration file, provided with the `--config` argument, as this will be needed to verify configuration. The file can be viewed with a command such as `more` or `less`, like so: